

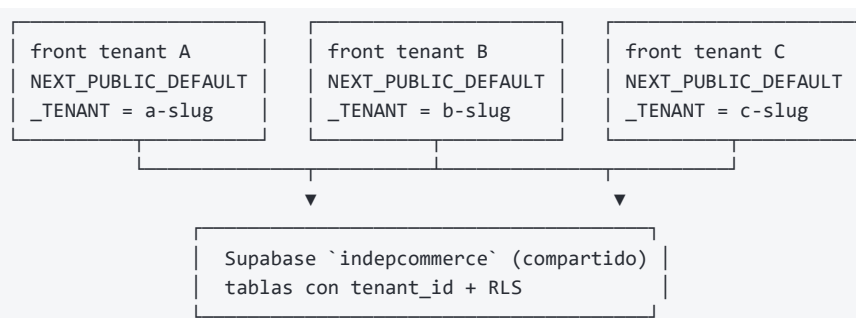
Guía de implementación de tenants — IndepCommerce

Backend multitenant **compartido** (Supabase, proyecto indepcommerce) + **un front por tenant**. Esta guía cubre el modelo, el esquema de tablas, cómo dar de alta un tenant, cómo agregar tablas específicas de tenant y las reglas para modificar el esquema sin romper el aislamiento.

Convención: el código, los nombres de tablas/columnas y el SQL van en inglés; las explicaciones en español. El SQL es copiable tal cual (ajusta los valores entre <...>).

1. Modelo / arquitectura

- **Un solo backend** multitenant: todas las tiendas viven en el mismo proyecto Supabase (cimqhmgbjhsdgbvmskmw). El aislamiento es por **tenant_id + RLS** (multitenancy *pooled*: tablas compartidas, no un schema/DB por tenant).
- **Un front por tenant**: cada tienda es su propio deploy de este template Next.js, "amarrado" a un tenant por variable de entorno (NEXT_PUBLIC_DEFAULT_TENANT). El front resuelve su tenant al iniciar y **filtra toda consulta por su tenant_id**.
- **La marca sale de la BD** (tenants.name, logo_url, theme): clonas el template, cambias el env y queda rebrandeado sin tocar código.



2. Esquema de la base de datos

Enums

Enum	Valores
currency_code	PEN, USD
order_status	pending, paid, shipped, delivered
tenant_role	owner, admin

Funciones

Función	Tipo	Para qué
set_updated_at()	trigger	Setea updated_at = now() en cada UPDATE.
handle_new_user()	trigger, security definer	Crea la fila profiles al insertarse un auth.users.
is_tenant_admin(p_tenant uuid)	security definer, stable	true si el usuario actual administra ese tenant. Se usa dentro de las políticas RLS.

Hardening aplicado: revoke execute de set_updated_at y handle_new_user a todos los roles;

is_tenant_admin revocada a anon (la mantiene authenticated porque las RLS la invocan).

Tablas

tenants — una fila por tienda.

Columna	Tipo	Notas	
id	uuid PK	gen_random_uuid()	
slug	text unique	usado en NEXT_PUBLIC_DEFAULT_TENANT	
name	text	marca (wordmark)	
custom_domain	text unique, null	dominio propio futuro	
logo_url	text, null	si está, el front usa ; si no, el SVG por defecto	
theme	jsonb	{ "accent": "#...", "accent2": "#..." }	
default_currency	currency_code	default PEN	
status	text	active \	suspended (default active)
created_at / updated_at	timestampz		

profiles — 1:1 con auth.users (nivel plataforma, NO tiene tenant_id).

Columna	Tipo	Notas
id	uuid PK -> auth.users(id)	cascade
full_name / phone	text, null	
created_at / updated_at	timestampz	

tenant_members — qué usuario administra qué tenant (M:N).

Columna	Tipo	Notas
tenant_id	uuid -> tenants(id)	PK compuesta
user_id	uuid -> auth.users(id)	PK compuesta, índice
role	tenant_role	default admin
created_at	timestampz	

products — catálogo (tenant-owned).

Columna	Tipo	Notas	
id	uuid PK		
tenant_id	uuid -> tenants(id)	NOT NULL, índice	
slug	text	unique (tenant_id, slug)	
name / description	text		
price	numeric(10,2)	>= 0	
currency	currency_code	default PEN	
material / collection	text, null		
compatibility	text[]	default {}	
is_featured	boolean	default false	
status	text	published \	draft (default published)
created_at	timestampz		

product_images / inventory — hijos de products (heredan el tenant vía el padre).

- product_images(id, product_id -> products, image_url, alt_text, display_order)
- inventory(id, product_id unique -> products, stock_level >= 0, updated_at)

orders — pedidos (tenant-owned).

Columna	Tipo	Notas
id	uuid PK	
tenant_id	uuid -> tenants(id)	NOT NULL, índice
user_id	uuid -> auth.users, null	set null (invitados = null)
status	order_status	default pending
total_amount	numeric(10,2)	>= 0
currency	currency_code	

Columna	Tipo	Notas
email / full_name / phone	text, null	
shipping_address	jsonb, null	
shipping_district / payment_provider / payment_reference	text, null	
created_at	timestampz	

order_items — líneas de pedido. order_id -> orders (cascade), product_id -> products (restrict), quantity > 0, price_at_purchase >= 0.

tenant_payment_config — credenciales Mercado Pago por tenant (**server-only**).

Columna	Tipo	Notas
tenant_id	uuid PK -> tenants(id)	
mp_access_token	text, null	SECRETO — solo se lee con service_role
mp_public_key	text, null	
updated_at	timestampz	

Relaciones

```

tenants 1——* tenant_members *——1 auth.users 1——1 profiles
|                                     |
| 1                                 (user_id, nullable)
*                                     *

products 1——* product_images      orders 1——* order_items *——1 products
| 1——1 inventory                    (tenant_id en orders y products)
|

tenants 1——1 tenant_payment_config

```

3. Reglas de multitenancy (invariantes)

1. **Toda tabla tenant-owned tiene tenant_id uuid not null references tenants(id) on delete cascade.**
2. **Unicidad por tenant**, no global: unique (tenant_id, <clave_natural>) (ej. (tenant_id, slug)).
3. **Índice en tenant_id** en cada tabla tenant-owned.
4. **RLS siempre activado** y con política de admin basada en is_tenant_admin(tenant_id).
5. **El front SIEMPRE filtra por tenant_id**. La publishable key compartida + las políticas de lectura pública (status = 'published') **no aíslan por tenant** — exponen lo publicado de *todos*. El aislamiento de cara al público lo garantiza el front, no la RLS.
6. **auth.users/profiles son de plataforma** (sin tenant_id). El vínculo usuario<->tenant es tenant_members.
7. **Secretos (tokens MP) nunca a anon/authenticated**: se leen solo con service_role en el server.

4. Cómo funciona la RLS (resumen)

- **Lectura pública** (anon + authenticated): tenants activos, products/product_images/inventory con status = 'published'. Cross-tenant (por eso el front filtra).
- **Escritura / gestión**: solo authenticated que sean is_tenant_admin(tenant_id) de esa fila.
- **orders**: las crea el server con service_role (bypassa RLS) para invitados; los admins ven/actualizan las de su tenant; un usuario ve las suyas (user_id = auth.uid()).
- **tenant_payment_config**: solo admins del tenant (gestión); el token se lee server-side con service_role.

5. Alta de un nuevo tenant (backend)

Ejecutar en el SQL editor de Supabase / vía MCP. Reemplaza <...>.

5.1 Crear el tenant

```
insert into public.tenants (slug, name, theme, default_currency, status)
values (
  '<slug>', -- ej. 'mi-tienda' (sin espacios, va en la URL/env)
  '<Nombre Visible>',
  '{"accent":"#<hex>","accent2":"#<hex>}':::jsonb,
  'PEN',
  'active'
);
```

5.2 Cargar catálogo (opcional, si lo tienes)

```
insert into public.products
  (tenant_id, slug, name, description, price, currency, collection, is_featured, status)
select t.id, v.slug, v.name, v.description, v.price, v.currency::currency_code,
       v.collection, v.is_featured, 'published'
from (select id from public.tenants where slug = '<slug>') t
cross join (values
  ('prod-1', 'Producto 1', 'Desc...', 99.00, 'PEN', 'categoria', true)
  -- ...más filas
) as v(slug, name, description, price, currency, collection, is_featured)
on conflict (tenant_id, slug) do nothing;

-- inventario inicial
insert into public.inventory (product_id, stock_level)
select p.id, 0 from public.products p
where p.tenant_id = (select id from public.tenants where slug = '<slug>')
and not exists (select 1 from public.inventory i where i.product_id = p.id);
```

5.3 Crear el usuario admin (owner)

Opción A — **vía signup en la app** (recomendado en prod): el dueño se registra y luego corres solo el insert en tenant_members (5.4).

Opción B — **por SQL** (bootstrap rápido): crea el usuario confirmado + identidad email.

```
with new_user as (
  insert into auth.users (
    instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
    created_at, updated_at, raw_app_meta_data, raw_user_meta_data,
    confirmation_token, recovery_token, email_change_token_new, email_change
  ) values (
    '00000000-0000-0000-0000-000000000000', gen_random_uuid(),
    'authenticated', 'authenticated',
    '<admin@email>', crypt('<password>', gen_salt('bf')), now(),
    now(), now(),
    '{"provider":"email","providers":["email"]}':::jsonb,
    '{"full_name":"<Nombre Admin>}':::jsonb,
    '', '', '', ''
  ) returning id, email
)
insert into auth.identities (provider_id, user_id, identity_data, provider,
                             last_sign_in_at, created_at, updated_at)
select nu.id::text, nu.id,
       jsonb_build_object('sub', nu.id::text, 'email', nu.email, 'email_verified', true),
       'email', now(), now(), now()
from new_user nu;
```

5.4 Vincular el usuario como owner del tenant

```
insert into public.tenant_members (tenant_id, user_id, role)
select t.id, u.id, 'owner'
from public.tenants t
join auth.users u on u.email = '<admin@email>'
where t.slug = '<slug>'
on conflict (tenant_id, user_id) do nothing;
```

5.5 Credenciales Mercado Pago del tenant

```
insert into public.tenant_payment_config (tenant_id, mp_access_token, mp_public_key)
select id, '<MP_ACCESS_TOKEN>', '<MP_PUBLIC_KEY>'
from public.tenants where slug = '<slug>'
on conflict (tenant_id) do update
  set mp_access_token = excluded.mp_access_token,
      mp_public_key    = excluded.mp_public_key,
      updated_at       = now();
```

6. Front de un nuevo tenant (deploy)

1. Clona este repo (o duplica el proyecto en Vercel).
2. Configura el entorno (.env.local / variables de Vercel):

```
NEXT_PUBLIC_SUPABASE_URL=https://cimqhmgbjhsdgbvmskmw.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=<publishable key del proyecto>
NEXT_PUBLIC_DEFAULT_TENANT=<slug>          # <-- amarra el front a su tenant
SUPABASE_SERVICE_ROLE_KEY=<service_role> # server-only: órdenes invitado + token MP
```

3. (Opcional) Ajusta defaults del template que **no** viven en la BD: src/config/site.ts (tagline, description, mainNav, socialLinks, url). El **nombre, logo y colores** sí salen del tenant.
4. Deploy. Verifica: / carga, /admin/login deja entrar al owner, el catálogo es solo de ese tenant.

Lo que toma de la BD automáticamente: getCurrentTenant() en

src/lib/tenant.ts resuelve el slug del env; la capa de datos

src/lib/data/products.ts filtra por tenant_id; el layout del

storefront aplica theme (colores) y marca.

7. Agregar una tabla específica de tenant

Regla de oro: aunque la tabla la use *un solo* tenant, créala como **tabla compartida con tenant_id** (pooled). No crees schemas ni tablas "por tenant" sueltas — mantiene el modelo uniforme, la RLS consistente y los tipos generados limpios. Si solo un tenant la usa, simplemente nadie más inserta filas.

7.1 Plantilla — tabla tenant-owned

```
create table public.<table> (
  id          uuid primary key default gen_random_uuid(),
  tenant_id   uuid not null references public.tenants (id) on delete cascade,
  -- ...columnas propias...
  status      text not null default 'published',    -- si es de cara al público
  created_at  timestampz not null default now(),
  updated_at  timestampz not null default now(),
  unique (tenant_id, <clave_natural>)              -- ej. (tenant_id, slug)
);
create index <table>_tenant_idx on public.<table> (tenant_id);

create trigger <table>_set_updated_at before update on public.<table>
  for each row execute function public.set_updated_at();

alter table public.<table> enable row level security;

-- Lectura pública SOLO si el storefront la muestra (recuerda: el front igual filtra por tenant_id):
create policy "Public can view published <table>"
  on public.<table> for select using (status = 'published');

-- Gestión por admins del tenant:
create policy "Tenant admins manage <table>"
  on public.<table> for all to authenticated
  using (public.is_tenant_admin(tenant_id))
```

```
with check (public.is_tenant_admin(tenant_id));
```

7.2 Plantilla — tabla hija (scoped vía el padre)

Si la tabla cuelga de otra tenant-owned (como `product_images` de `products`), **no** repitas `tenant_id`: deriva el `tenant` del padre en la RLS.

```
create table public.<child> (  
  id          uuid primary key default gen_random_uuid(),  
  <parent>_id uuid not null references public.<parent> (id) on delete cascade,  
  -- ...columnas...  
);  
create index <child>_<parent>_idx on public.<child> (<parent>_id);  
  
alter table public.<child> enable row level security;  
  
create policy "Public can view <child> of published <parent>"  
  on public.<child> for select using (  
    exists (select 1 from public.<parent> p  
      where p.id = <parent>_id and p.status = 'published')  
  );  
  
create policy "Tenant admins manage <child>"  
  on public.<child> for all to authenticated  
  using (exists (select 1 from public.<parent> p  
    where p.id = <parent>_id and public.is_tenant_admin(p.tenant_id)))  
  with check (exists (select 1 from public.<parent> p  
    where p.id = <parent>_id and public.is_tenant_admin(p.tenant_id)));
```

7.3 Tabla con secreto (como `tenant_payment_config`)

- 1:1 con `tenant`: `tenant_id` uuid primary key references `tenants(id)`.
- RLS: **solo** política de admins (for all to authenticated using `is_tenant_admin`). **Sin** lectura pública.
- El valor secreto se lee server-side con `service_role` (ver `src/lib/supabase/admin.ts`).

8. Reglas de modificación del esquema

1. **Todo cambio de esquema va por migración** (`apply_migration`, nombre en `snake_case`). Nunca edites tablas a mano sin dejar el SQL en `supabase/`.
2. **No quites** `tenant_id`, sus FKs/índices, ni desactives RLS de una tabla tenant-owned.
3. **Después de cualquier DDL:**
 - Regenera tipos TS -> `generate_typescript_types` y guárdalos en `src/types/database.types.ts`.
 - Corre `get_advisors` (security y performance) y resuelve lo que aparezca.
 - `npx tsc noEmit` debe quedar en 0.
4. **Funciones SQL:** security definir solo cuando haga falta; siempre `set search_path = ''`; `revoke execute ... from anon` salvo que `anon` deba llamarla; mantenla en `authenticated` solo si una política RLS la usa.
5. **Cambios destructivos** (drop/rename de columnas usadas por el front): primero actualiza el front y los tipos, luego aplica el DDL, para no dejar el build rojo.
6. **Datos sembrados** (DML) van con `execute_sql`, no `apply_migration`. No hardcodees IDs generados: resuélvelos por slug/clave natural.

9. Checklist de seguridad (por tabla nueva)

- [] `tenant_id` NOT NULL + FK on delete cascade (o derivado del padre).

- [] enable row level security.
- [] Política de gestión con is_tenant_admin(tenant_id) (using **y** with check).
- [] Lectura pública solo si corresponde, y acotada (ej. status = 'published').
- [] Secretos: sin acceso a anon/authenticated; lectura server-side con service_role.
- [] get_advisors (security) sin hallazgos nuevos.
- [] El front filtra por tenant_id en cada query de esa tabla.

10. Referencia rápida (archivos del repo)

Archivo	Rol
supabase/indepcommerce/01_schema.sql	Esquema (tablas, enums, funciones, hardening)
supabase/indepcommerce/02_rls.sql	Políticas RLS
src/lib/tenant.ts	getCurrentTenant, getTenantBySlug, tenantBrand, tenantThemeVars, getTenantPaymentConfig
src/lib/supabase/admin.ts	Cliente service_role (server-only)
src/lib/data/products.ts	Capa de datos (filtra por tenant_id)
src/lib/auth.ts	requireAdmin (valida tenant_members)
src/app/api/checkout/mercadopago/route.ts	Checkout per-tenant + orden de invitado